



WebTestPilot: Agentic End-to-End Web Testing against Natural Language Specification by Inferring Oracles with Symbolized GUI Elements

Jinhyeok Choi

2026.06.22

Base Lab @DGIST



Overview

1. Background & Motivation

2. Motivating Example

3. Approach

4. Benchmark Construction

5. Experiments

6. Real-World Case Study

7. Discussion

Background & Motivation

The global progressive web application market is projected to reach USD **9.4 billion** by 2030.
As web apps grow, the importance of **E2E (End-to-End) testing** grows with them.

Exploration-based Testing

This paper →

Explores all possible web states.

Ex) Random / Model-based / Search-based

Specification-based Testing

Verifies whether behavior is consistent with business requirements.

Converts any natural language source (PRDs, UX specs, QA plans) into executable tests.

Background & Motivation

Motivation: State of Software Quality Report 2024: over 58% of respondents use LLM-based tools for automated testing, yet —



Capability
gaps



Reliability
concerns

Background & Motivation

When an agent flags an inconsistency, it is unclear whether the issue stems from the **agent itself (hallucination)** or the **web application (a real bug)**.

→ To resolve this ambiguity, the paper defines two core technical challenges

1. Implicit Oracle Inference

Must independently judge independencies between states that are never explicitly stated

2. Probabilistic Inference

The non-deterministic nature of LLMs makes verification results unstable

Background & Motivation

1. Implicit Oracle Inference

"An item was added to the cart" — verifying this requires comparing not just the current screen, but also **prior states (searched items, quantity, etc.)**

Limitations of Existing LLM-Based Approaches (Limited Oracles)

LaVague / NaviQAte

Treats reaching the end
of the flow as success

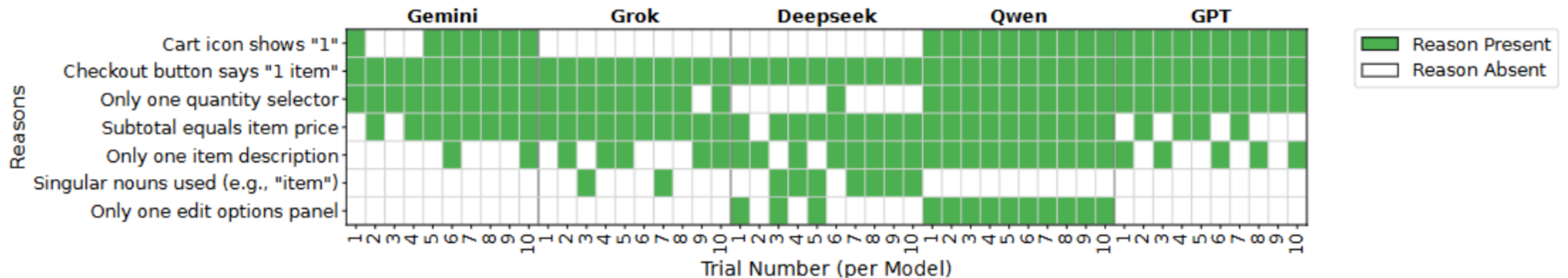
PinATA

Verifies each state in isolation,
ignoring cross-state context

Background & Motivation

2. Probabilistic Inference

Even with fixed model settings, the same prompt can yield **different responses**.



Background & Motivation

2 Insight about WebTestPilot



Insight 1

Deep Contextual Understanding

Capturing implicit expectations requires a deep contextual understanding of dependencies between states.

Background & Motivation

2 Insight about WebTestPilot



Insight 2

Combining Neural + Symbolic Reasoning

To gain both flexibility (LLM agent) and stability (formal assertion), neural reasoning must be combined with symbolic reasoning.

Motivating Example

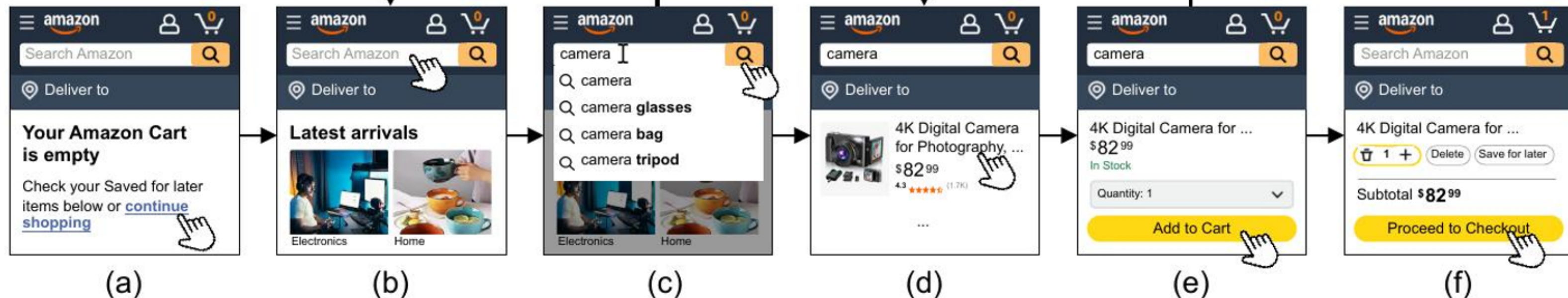
Three Dependencies in an Amazon Cart Flow

Causal dependency

After typing in search bar in (c), the background becomes darker, the placeholder is replaced by the query, a dropdown appears below the search bar, showing suggestions that contain the query.

Data dependency

In (e), the product title and price match those of the first listing in (d)



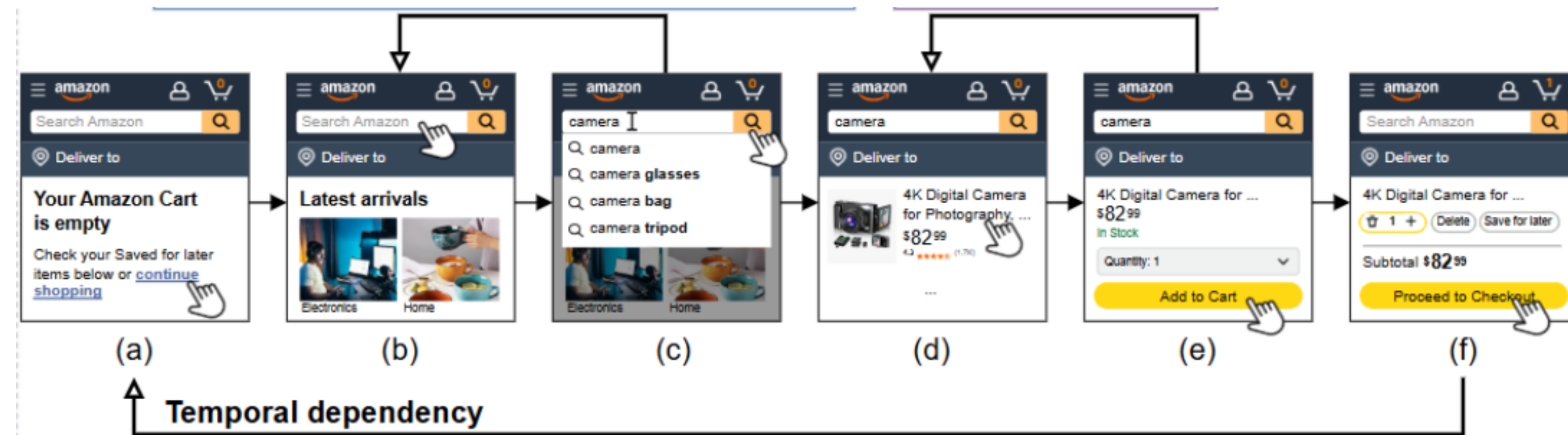
Temporal dependency

Compared to (a), the shopping cart icon in (f) displays a count incremented by 1

The checkout button and subtotal appear in (f) but are absent in (a)

The empty cart message shown in (a) is no longer present in (f)

Motivating Example



(a) Cart Page

Start with an empty shopping cart
Click "Continue Shopping"

(b) Home Page

Navigate to the homepage
Enter "camera" in the search bar

(c) Search Suggestions

View the autocomplete suggestions
Submit the search query

(d) Search Results

Open the search results page
Select the first product

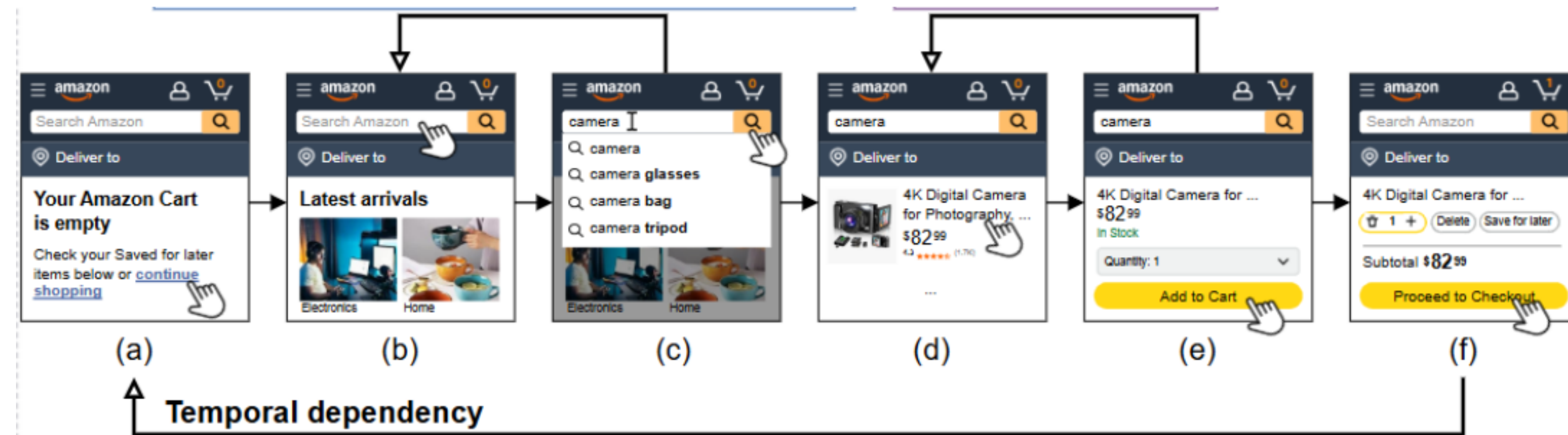
(e) Product Details

View the product details
Click "Add to Cart"

(f) Cart Verification

Verify that the selected product has been successfully added to the cart And Complete the test flow

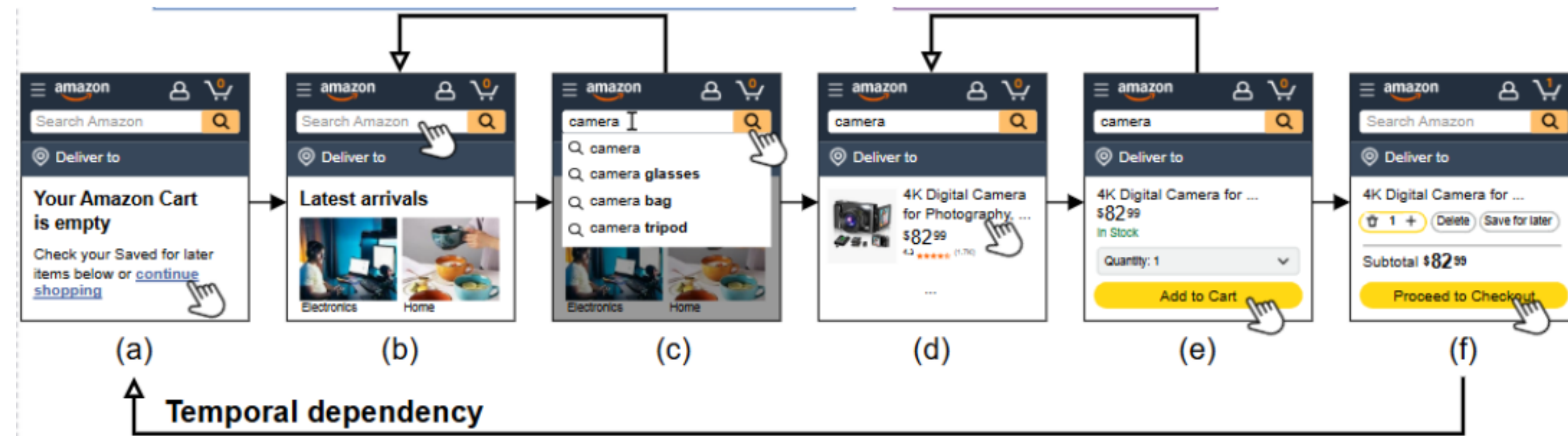
Motivating Example



1. Causal Dependency

In (b), the search bar must be focused before typing. Typing "camera" transitions to (c), where each suggestion should include the typed query from (b) as a substring.

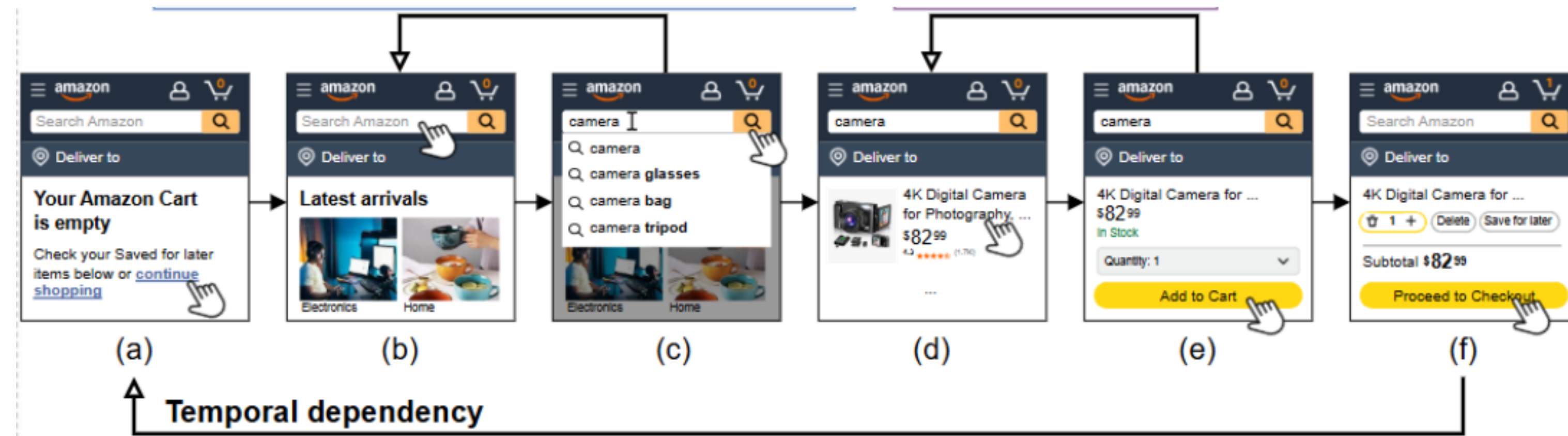
Motivating Example



2. Data Dependency

The checkout summary in (f) must match the items, quantities, and prices selected in (e). Similarly, the product in (e) must correspond to a result in (d)

Motivating Example



3. Temporal Dependency

States (a) and (f) are the same logical page (cart). Compared to (a), the cart in (f) reflects updated counts, displays the checkout and subtotal, and removed the empty-cart message.

Motivating Example

Symbolization: Abstracting State into Variables

To enable robust cross-state verification of implicit and explicit expectations, WebTestPilot supports **declarative schemas**, which act as **symbol templates** (or “variables”) representing structured UI data.

1. WebTestPilot applies a **symbolization stage** to define relevant symbols.
2. WebTestPilot then instantiates these schemas with values extracted from the **current and prior states**.
3. verify that the post-action state satisfies both **explicit expectations** and **implicit expectations** derived from prior states.

Motivating Example

Symbolization: Abstracting State into Variables

First, WebTestPilot applies a **symbolization stage** to define relevant symbols (i.e., Cart and Product)

```
class Product(BaseModel):  
    title: str = Field(...)  
    price: float = Field(..., ge=0)  
    quantity: Optional[int] = Field(None, gt=0)  
  
class Cart(BaseModel):  
    items: List[Product] = Field(...)
```

Motivating Example

Symbolization: Abstracting State into Variables

Second, WebTestPilot then instantiates these schemas with values extracted from the **current and prior states** (i.e., state (f) and (e))

```
# (e) Extract added product from the latest state
added = session.history[-1].extract("Get product detail", schema=Product)

# (f) Extract current cart summary
current = session.state.extract("Get cart summary", schema=Cart).items

# (g) Extract previous cart summary
prior = session.history[-2].extract("Get cart summary", schema=Cart).items
```

Motivating Example

Symbolization: Abstracting State into Variables

Third, verify that the post-action state satisfies both **explicit expectations** and **implicit expectations** derived from prior states

```
# Verify products (title, quantity, price) consistency
for prod in prior + [added]:
    match = next((p for p in current if p.title == prod.title), None)
    assert match is not None, f"Product {prod.title} missing in current cart"
    assert match.quantity == prod.quantity, f"Quantity mismatch for {prod.title}"
    assert match.price == prod.price, f"Price mismatch for {prod.title}"

# Verify subtotal consistency
prior_subtotal = sum(p.price * p.quantity for p in prior)
added_total = added.price * added.quantity
current_subtotal = sum(p.price * p.quantity for p in current)
assert current_subtotal == prior_subtotal + added_total, "Cart subtotal mismatch"
```

Approach

Problem Statement

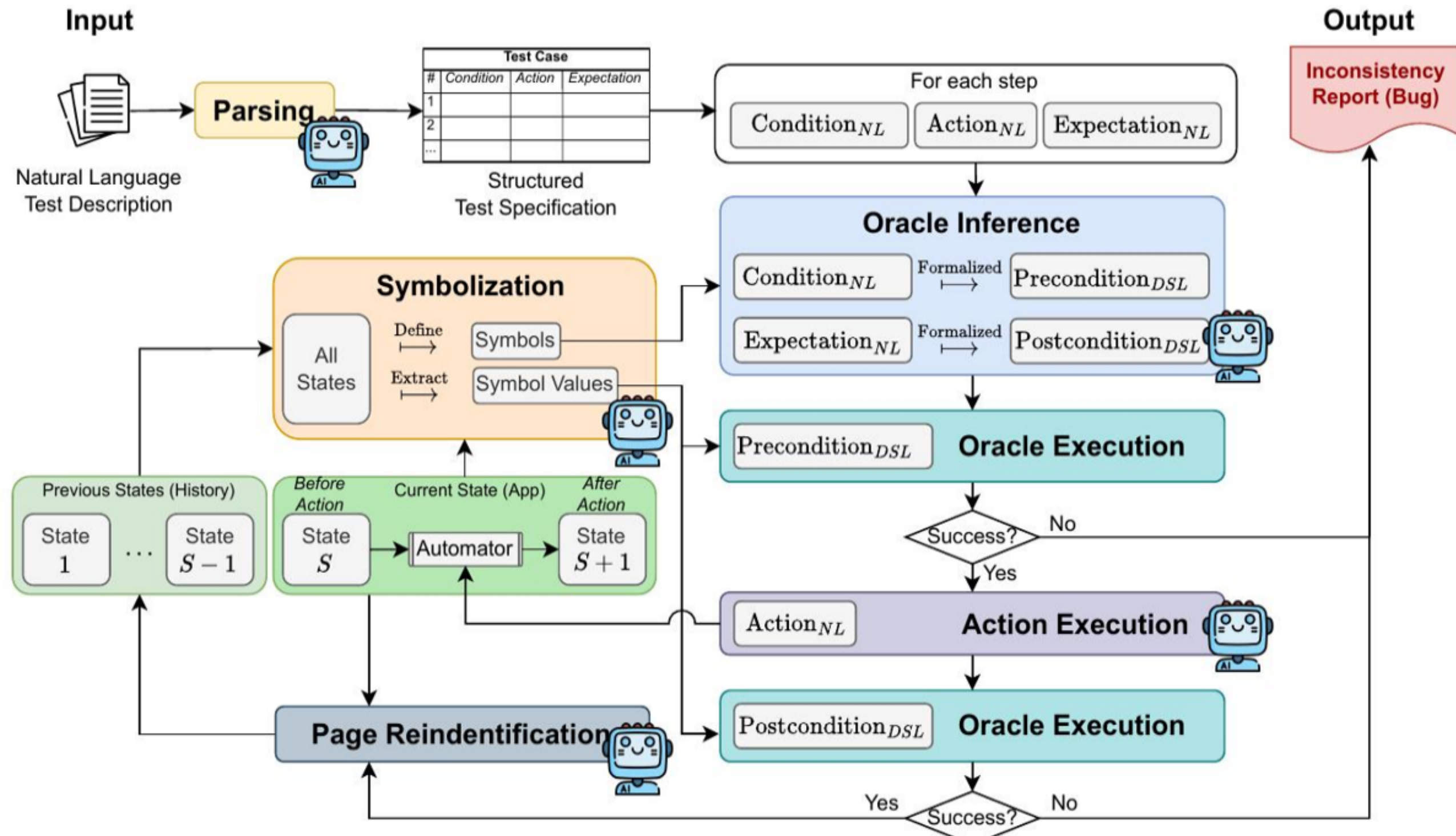
Modeling the Web App W = a graph of states s . Applying an action $a = \langle t, e, \text{element} \rangle$ (type, target element, parameters) causes a state transition $T: (s, a) \mapsto s'$.
Applying a sequence of actions produces an execution trace $\tau = s_0 \rightarrow a_1 s_1 \rightarrow a_2 \cdots \rightarrow a_n s_n$

The Role of Tester T

- 1 Parse the natural language requirement D into a sequence of steps: $D = \text{step}_1 \oplus \cdots \oplus \text{step}_n$
- 2 Map and execute each step as an action: $a_i = T(\text{step}_i)$, transitioning to a new state s_i
- 3 Generate a predicate $p_i : S \rightarrow \{\top, \perp\}$ for each new state

Bug definition: $s_i \not\models p_i$ — i.e., state s_i is inconsistent with the requirement of step_i

Approach



1. Input Parsing
Natural Language $\rightarrow$ Structured sequence of steps
(condition, action, expectations)

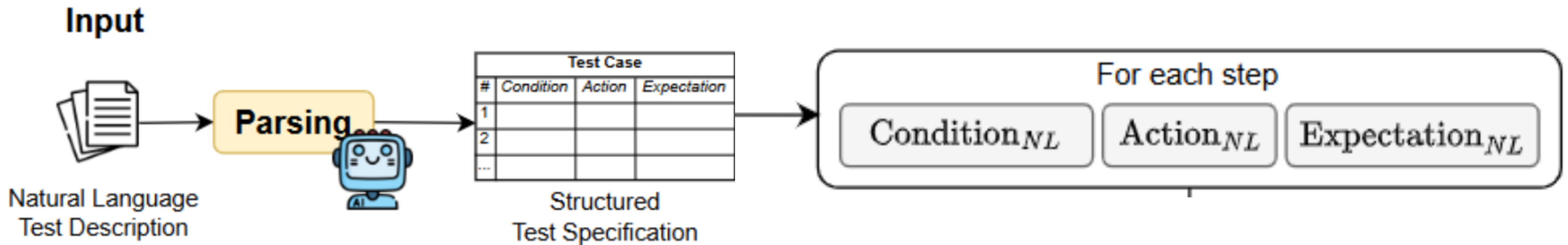
2. Oracle Inference
Identify dependencies from state history $\rightarrow$ define
symbols $\rightarrow$ generate pre/postcondition assertions in
DSL

3. Oracle Execution
Check precondition $\rightarrow$ execute action $\rightarrow$ check
postcondition. Retry on failure; persistent failures
are logged as bugs

Approach

1. Input Parsing

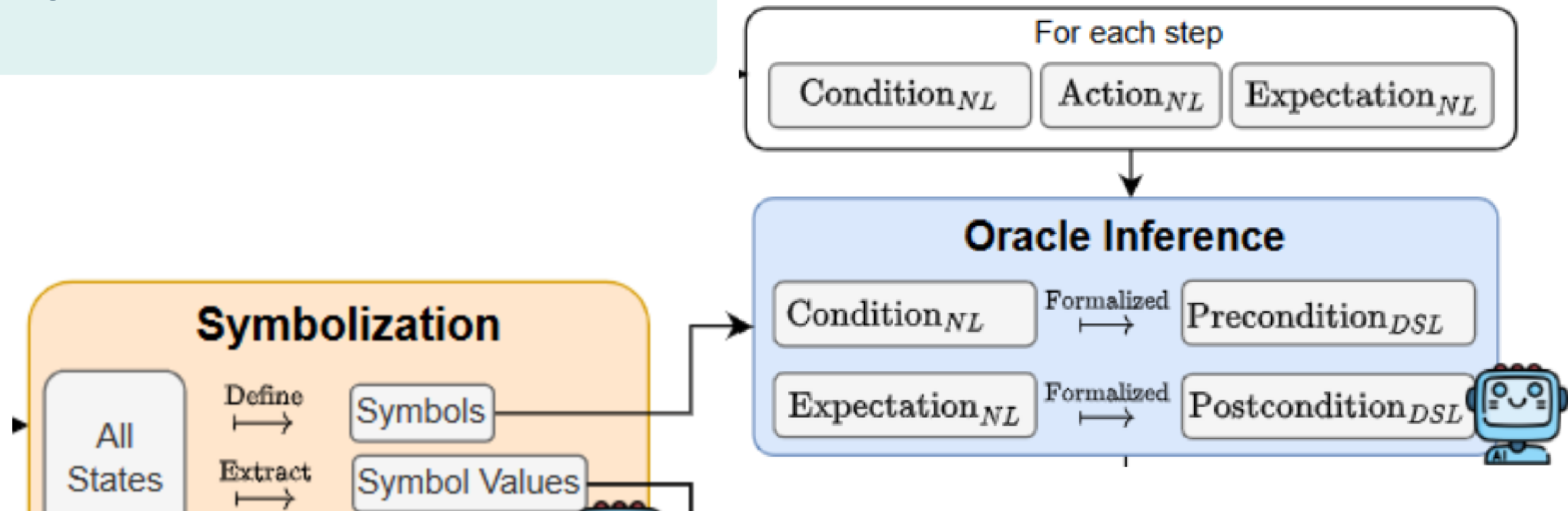
Natural Language → Structured sequence of steps
(condition, action, expectations)



Approach

2. Oracle Inference

Identify dependencies from state history → define symbols → generate pre/postcondition assertions in DSL



Approach

Built-in DSL symbols, their attributes and methods

Type	Methods / Attributes	Return Type	Description
Session	history state	list[State] State	Chronological list of all states. Current browser page.
State	page_id page_title page_url elements find(description: str, top_k: int) extract(instruction: str, schema: BaseModel)	string string string set[Element] list[Element] BaseModel	Logical page identifier shared across states. Title of active tab. Current state's page URL. All state elements (flattened). Top K elements matching description (may be empty). Extracts a schema-conforming symbol from the state.
Element	id text is_visible is_dynamic xmin, ymin, xmax, ymax parent children observe(visual_status: str)	string string bool bool int Element list[Element] bool	Unique element identifier. Visible text content. Visible in viewport. Structure varies with siblings. Bounding box coordinates. Parent element. Child elements. Check visual consistency.

Approach

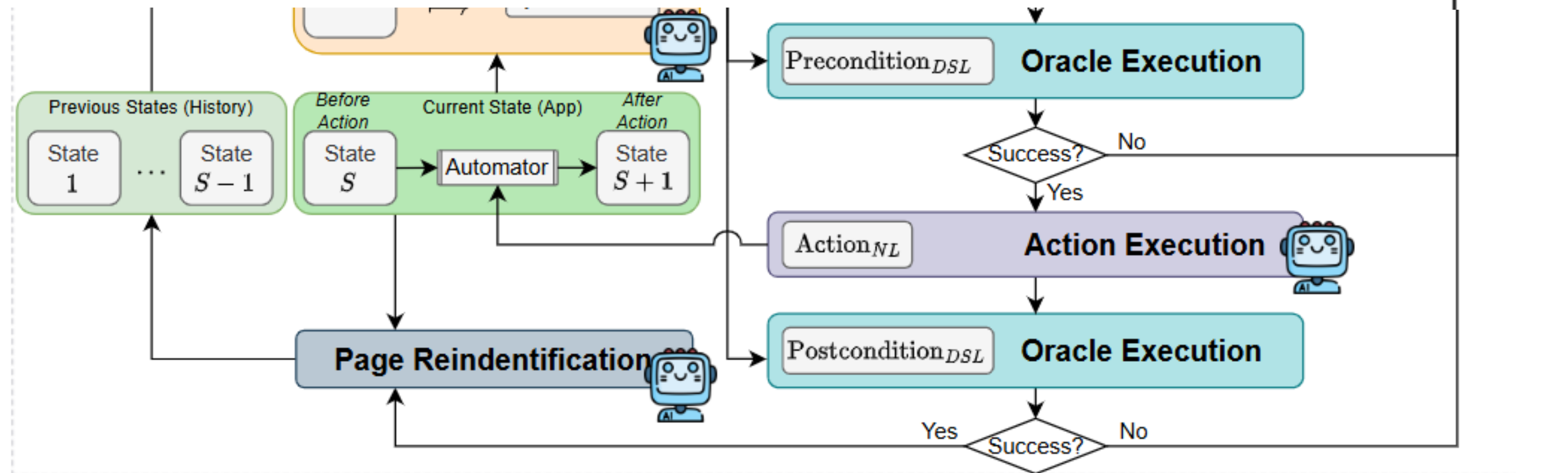
5 Assertion Types

- **Existence** `state.find("profile") is not None`
- **Relational** `checkout_btn.ymin > cart_icon.ymax`
- **Temporal** `countdown value decreases across states in order`
- **Causal** `cart item count delta == 1`
- **Data Integrity** `subtotal ==  $\Sigma$ (item.price)`

Approach

3. Oracle Execution

Check precondition → execute action → check postcondition. Retry on failure; persistent failures are logged as bugs



Approach

Oracle Execution

1. Action Execution

ActionNL + screenshot → the LLM infers the **action type** (click/type/drag/scroll/wait), target description, and parameters

3. Retry on Failure

On assertion failure, regenerates and retries up to n times.

2. Page Reidentification

Assigns a **page_id** to each new state.

Compares the layout against prior states, then uses the LLM to judge whether the closest candidate is the same page before reusing or assigning a new ID.

Benchmark Construction

1. Select Candidate Apps

2. Write Test Scripts

3. Map NL ↔ Scripts

4. Build Interaction Recorder

4 Web Application

Popularity (★5,000+) · Active development (50+ contributors, 1,000+ commits) etc.

Web Application	Category	Tech Stack	LOC	# Test Scripts	# Injected Bugs			
					NA	NOOP	NAV	EXEC
Bookstack	Knowledge Base	PHP (Laravel)	214,819	26	8	8	3	7
Indico	Event Management	Python (Flask) + React	573,316	31	14	3	3	10
Invoiceninja	Invoicing	PHP (Laravel) + Vue.js	1,513,289	32	0	4	3	20
Prestashop	E-Commerce	PHP (Symfony) + Vue.js	2,234,514	23	7	6	4	6

Benchmark Construction

Types of injected bugs in the benchmark

Bug Category	Bug	Code	Description	Example
Visual	Missing UI Element	NA	This bug refers to the absence of essential user interface components on the page, which prevent user interaction, disrupt user workflows, and cause core functionality to fail.	(indico/#239) "Send" button is missing from request recording in lectures.
Functional	Operation No Response	NOOP	This bug occurs when a user action triggers no visual feedback or system response. The lack of acknowledgment breaks the user's mental model of the application's expected behavior, causing confusion, frustration, and potentially leading to task abandonment.	(bookstack/#5323) No error message shown when user does not have permission to delete attachment.
	Navigation Logic Error	NAV	This bug arises when user interactions redirect or transition the application to unintended or illogical pages, violating the expected workflow or business rules. It can mislead users and disrupt tasks.	(prestashop/#39044) Clicking a product in "All Stores" send you to the "Order" page not the "Edit Product" page.
	Unexpected Task Result	EXEC	This bug occurs when the output or state resulting from an operation does not match the expected specifications or requirements. This discrepancy may be in data content, displayed information, or other quantitative results.	(invoiceninja/#10351) Generating a PDF statement for a client shows the wrong client name and address.

Experiments

Evaluation Metrics

$$\text{Trace}_x = \langle tr_1, tr_2, \dots, tr_n \rangle, \quad \text{where } tr_i = (url_i, action_i, target_i)$$

- Task Completion(TC)

$$\text{TC}_x^T = \begin{cases} 1 & \text{if } \text{LCS}(\text{Trace}_x^T, \text{Trace}_x^*) == \text{Trace}_x^* \\ 0 & \text{otherwise} \end{cases}$$

1 if the ground-truth trace is a subsequence of the agent's trace, else 0

- Correct Trace(CT)

$$\text{CT}_x^T = \frac{|\text{LCS}(\text{Trace}_x^T, \text{Trace}_x^*)|}{|\text{Trace}_x^*|}$$

LCS length normalized by ground-truth trace length — measures "how far" progress went even on failure

Experiments

Baseline Models

LaVague

Multi-agent Most popular open source

Two-stage architecture:
World Model + **Action Engine**
Uses HTML DOM + screenshots.

Focuses solely on completing **navigation flows**, without verifying pre- or postconditions

NaviQAte

Single-agent

Action Planning (RAG) →
Choice Extraction (extract & rank elements) → **Decision Making** (LLM selects via annotated screenshot)

Like LaVague, NaviQAte focuses on test flow completion and does not handle verification or assertion tasks

PinATA

Multi-agent · SOTA

Three-agent division of labor:
Orchestrator (manages flow) +
Actor (UI actions) + **Assertor**
(verifies results via visual analysis), with shared long-term memory

PinATA's Assertor verifies only the conditions **explicitly stated in the test requirements**.

Experiments

RQ1. Test Flow Completion

How effectively does WebTestPilot generate test trajectories that align with human-authored test scripts, compared to baseline GUI testing agents?

Approach	Test Flow Completion			
	Task Completion (TC)	Correct Trace (CT)	Avg. Runtime (s/step)	Avg. Tokens (k/step)
NAVIQATE [52]	0.10	0.64	145	54.6
LAVAGUE [25]	0.50	0.75	38	8.6
PINATA [10]	0.25	0.71	48	19.6
WEBTESTPILOT	0.84	0.86	10	7.7

WebTestPilot achieves the highest **completion rate** (0.84) and most **correct traces** (0.86) — +14.7pp over the strongest baseline, while also using the least runtime and tokens

Experiments

RQ2. Bug Detection

How effective is WebTestPilot at detecting visual and functional faults during GUI testing, relative to existing agent-based baselines?

Approach	Bug Detection	
	Precision (%)	Recall (%)
NAVIQATE [52]	–	–
LAVAGUE [25]	–	–
PINATA [10]	0.50	0.26
WEBTESTPILOT	0.86	0.80

Precision

$$\frac{\text{\# of correct violations}}{\text{\# of total violations}} \quad \textit{Precision} = \frac{TP}{TP+FP}$$

Recall

$$\frac{\text{\# of correct violations}}{\text{\# of injected bugs}} \quad \textit{Recall} = \frac{TP}{TP+FN}$$

Experiments

RQ3. Robustness Evaluation

To what extent can WebTestPilot generate correct test cases when provided with requirements expressed in a varied, unstructured, or **freely written natural language**, without relying on a fixed input format?

Original Text

Click "Books" link in navigation. Books listing page appears. Verify "Create New Book" link is visible [...]

Add Noise

frmo tr dsboard amd tpa-
pin9 the "Boosk" llnk nes-
tled withi he naviatno
mneu [...]

Dropout

Redacted. Books listing
page appears. Verify "Cre-
ate New Book" link is vis-
ible [...]

Restyle

To begin your journey
through the digital li-
brary, start by navigating
[...]

Summarize

From dash, click 'Books'
-> verify 'Create New
Book' link -> click it ->
form opens [...]

Experiments

RQ4. Model Comparison

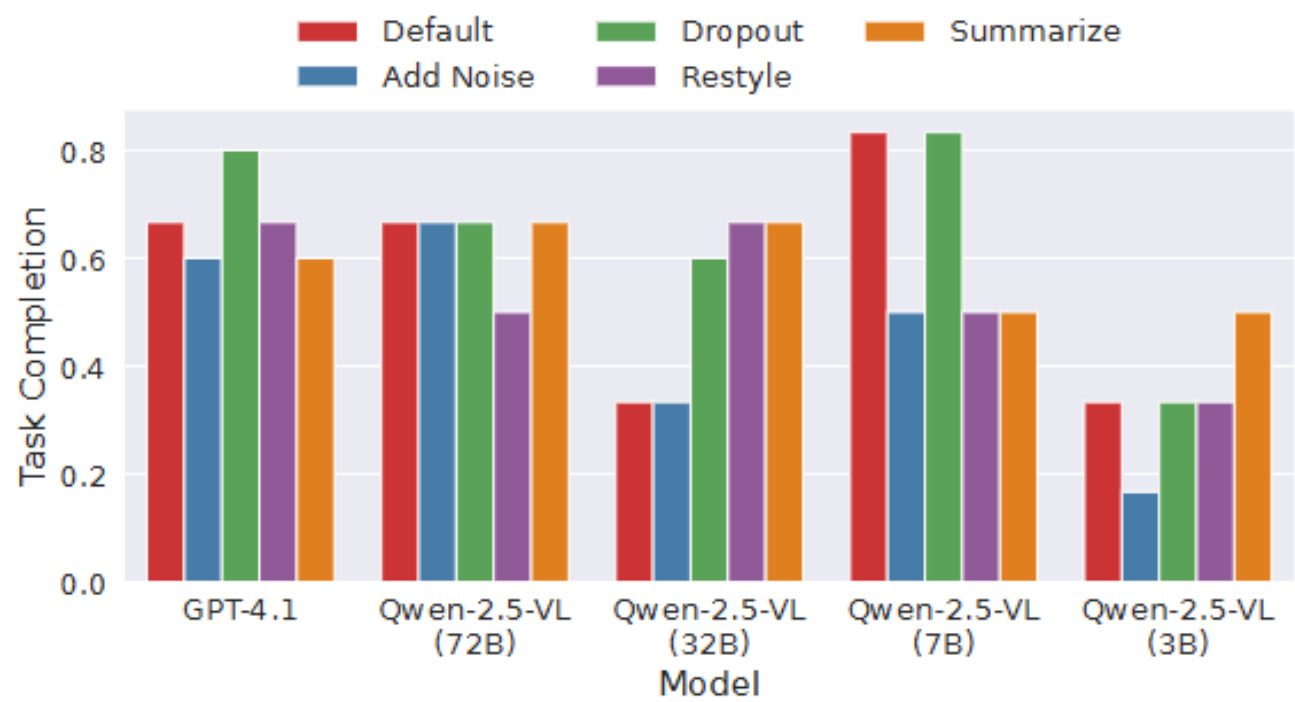
How much does WebTestPilot 's overall performance depend on the capabilities of the underlying language model, and to what extent can its refinement mechanisms compensate when using a lightweight and cost-effective LLM?

Model	Default		Add Noise		Dropout		Restyle		Summarize	
	TC	CT	TC	CT	TC	CT	TC	CT	TC	CT
GPT-4.1	<u>0.67</u>	0.85	<u>0.60</u>	<u>0.75</u>	<u>0.80</u>	0.83	0.67	0.85	<u>0.60</u>	<u>0.75</u>
Qwen2.5-VL-72b	<u>0.67</u>	<u>0.69</u>	0.67	0.92	0.67	<u>0.84</u>	0.50	0.69	0.67	0.92
Qwen2.5-VL-36b	0.33	0.46	0.33	0.69	0.33	0.69	<u>0.67</u>	<u>0.84</u>	0.67	0.54
Qwen2.5-VL-7b	0.83	0.85	0.50	0.61	0.83	0.85	0.50	<u>0.84</u>	0.50	0.54
Qwen2.5-VL-3b	0.33	0.46	0.17	0.31	0.17	0.31	0.33	0.61	0.50	0.54

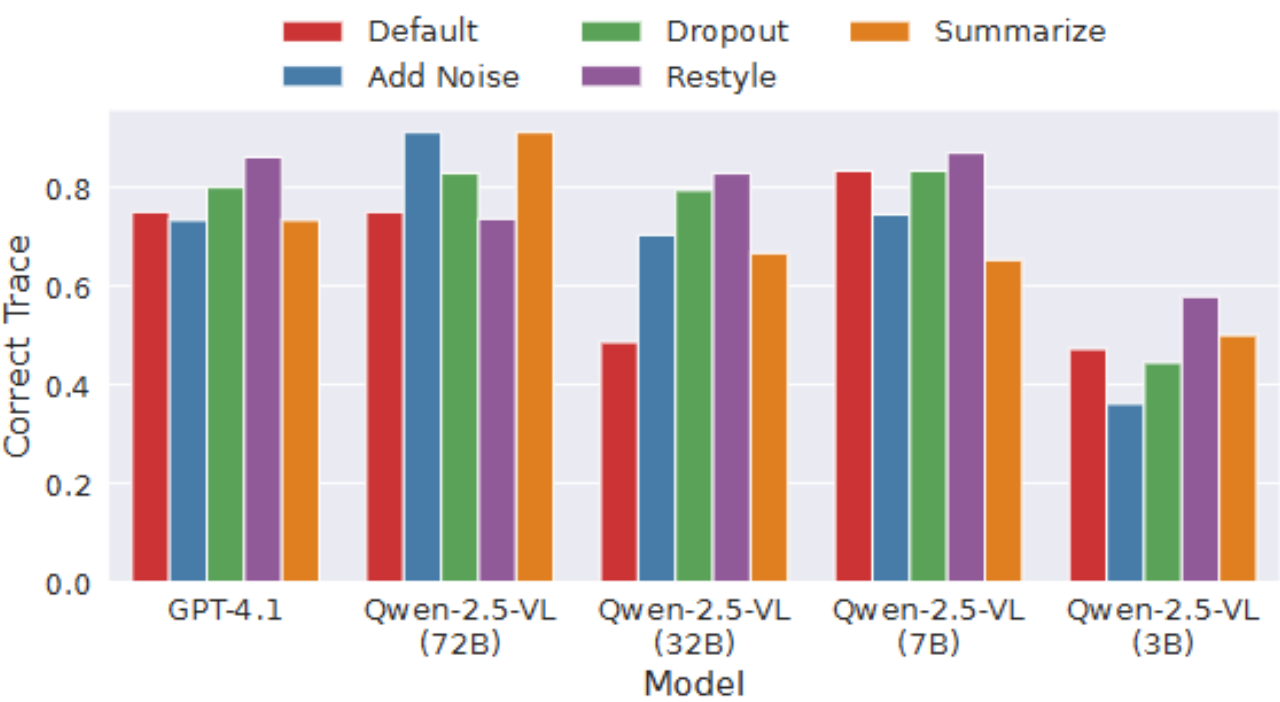
1. No Single Transformation Dominates Across Models
2. Failures Stem from Parsing, Not Execution
3. Bigger Models Aren't Always Better

Experiment Result

Model	Default		Add Noise		Dropout		Restyle		Summarize	
	TC	CT	TC	CT	TC	CT	TC	CT	TC	CT
GPT-4.1	<u>0.67</u>	0.85	<u>0.60</u>	<u>0.75</u>	<u>0.80</u>	0.83	0.67	0.85	<u>0.60</u>	<u>0.75</u>
Qwen2.5-VL-72b	<u>0.67</u>	<u>0.69</u>	0.67	0.92	0.67	<u>0.84</u>	0.50	0.69	0.67	0.92
Qwen2.5-VL-36b	0.33	0.46	0.33	0.69	0.33	0.69	<u>0.67</u>	<u>0.84</u>	0.67	0.54
Qwen2.5-VL-7b	0.83	0.85	0.50	0.61	0.83	0.85	0.50	<u>0.84</u>	0.50	0.54
Qwen2.5-VL-3b	0.33	0.46	0.17	0.31	0.17	0.31	0.33	0.61	0.50	0.54



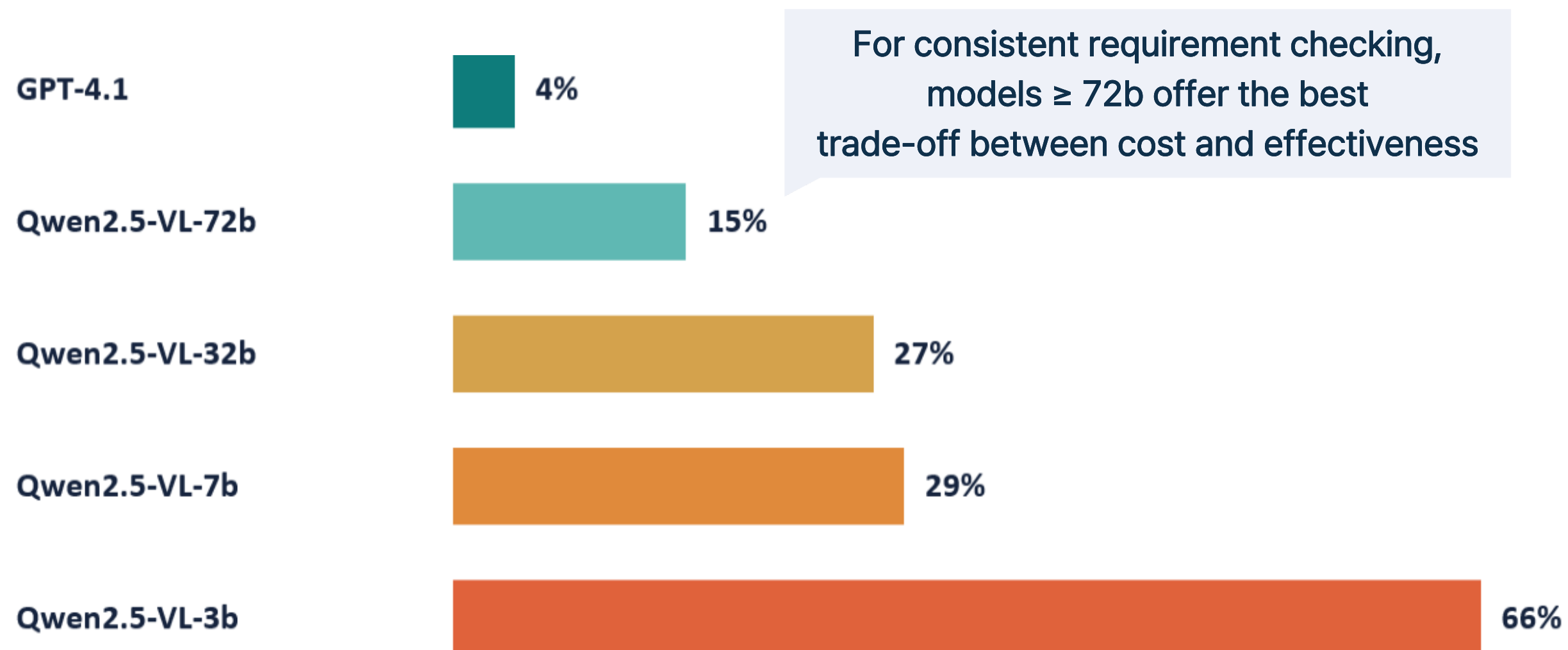
(a) Task Completion



(b) Correct Trace

Experiments

Assertion Failure



Small models ($\leq 7B$)

Use of undefined symbols

Mid-size models (7-32B)

Method/attribute hallucination

Large models ($\geq 32B$)

Assertion misuse (semantic errors)

Summary

Navigation favors smaller models, assertions favor larger models

Real-World Case Study

A collaboration with a no-code platform partner — finding 8 real bugs in a live warehouse management system under development

Data Binding 62.5%
UI 25%
Navigation 12.5%

Table 6. Bugs detected by WEBTESTPILOT in the case study.

#	Section	Page	Feature/Action	Bug Type	Bug Description
1	Warehouses	Warehouse Info	Table	Data	Some required fields in the table are empty.
2		Storage Area	Create	UI	Duplicate "Warehouse Name" form fields.
3		Storage Unit	Search	Data	Dropdown options for "Warehouse" is inconsistent with available warehouses.
4	Receipts	Receipt Info	Table	Nav	Clicking "Details" leads to an error page.
5	Assets	Inventory	Search Form	Data	Dropdown options for "Warehouse" are not bound to the actual table data.
6		Inventory	Search Form	Data	Dropdown options for "Storage Area" are not bound to the actual table data.
7		Asset	Search Form	Data	Dropdown options for "Supplier" is empty.
8	Device Management	Cameras	Search Form	UI	Query field names are incorrect.

But, **PinATA** is limited to UI and navigation issues (3/8 categories), and can only catch data issues **when explicitly specified**

Discussion

Threats to Validity

Internal: metrics may underestimate tester performance, as multiple paths can achieve the same functionality

External: benchmark (webapp, test case, bugs)
may not fully reflect real-world scenarios

A final **limitation** is that we assume requirements are self-contained and complete, specifying all conditions, actions, and expected outcomes in order.

Semantic Parsing for specification-Based Testing

experiments show that input parsing strategy and quality drive performance.

But Natural language requirements are often ambiguous, incomplete, or context-dependent.



Thank you!

WebTestPilot:
Agentic End-to-End Web Testing against Natural Language
Specification by Inferring Oracles with Symbolized GUI
Elements

Base Lab @DGIST